

UNIT-III

	UNIT-III	
	Functions: • Standard Mathematical Functions Input/Output Functions: • Unformatted & Formatted I/O function in C • Input Functions viz.getch() • Getche() • Gets() • Output Functions viz.	
	• Putch() • Putchar() • Puts() • String Manipulation Functions User Defined Functions: • Introduction/Definations • Prototype • Local and Global Variables • Passing Parameters • Recursion	

UNIT-III FUNCTIONS

Standard Mathematical Functions:

C Programming allows us to perform mathematical operations through the functions defined in <math.h> header file. The <math.h> header file contains various methods for performing mathematical operations such as sqrt(), pow(), ceil(), floor() etc.

C Math Functions

There are various methods in math.h header file. The commonly used functions of math.h header file are given below.

No.	Function	Description
1)	ceil(number)	rounds up the given number. It returns the integer value which is greater than or equal to given number.
2)	floor(number)	rounds down the given number. It returns the integer value which is less than or equal to given number.
3)	sqrt(number)	returns the square root of given number.
4)	pow(base, exponent)	returns the power of given number.
5)	abs(number)	returns the absolute value of given number.

C Math Example

Let's see a simple example of math functions found in math.h header file.

1. `#include<stdio.h>`
2. `#include <math.h>`
3. `int main(){`
4. `print f("\n%f",ceil(3.6));`
5. `print f("\n%f",ceil(3.3));`
6. `print f("\n%f",floor(3.6));`

7. `print f("\n%f",floor(3.2));`
8. `print f("\n%f",sqrt(16));`
9. `print f("\n%f",sqrt(7));`
10. `print f("\n%f",pow(2,4));`
11. `print f("\n%f",pow(3,3));`
12. `print f("\n%d",abs(-12));`
13. `return 0;`
14. `}`

Output:

```
Backward Skip 10sPlay VideoForward Skip 10s
4.000000
4.000000
3.000000
3.000000
4.000000
2.645751
16.000000
27.000000
12
```

Input/Output:

Unformatted & Formatted I/O Function in C:

Formatted I/O Functions

Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers. These I/O supports all data types like int , float, char, and many more.

Why they are called formatted I/O?

These functions are called formatted I/O functions because we can use format specifiers in these functions and hence, we can format these functions according to our needs.

List of some format specifiers-

S NO.	Format Specifier	Type	Description
1	%d	int /signed int	used for I/O signed integer value

2	%c	char	Used for I/O character value
3	%f	float	Used for I/O decimal floating-point value
4	%s	string	Used for I/O string/group of characters
5	%ld	long int	Used for I/O long signed integer value
6	%u	unsigned int	Used for I/O unsigned integer value
7	%i	unsigned int	used for the I/O integer value
8	%lf	double	Used for I/O fractional or floating data
9	%n	print s	print s nothing

The following formatted I/O functions will be discussed in this section-

1. printf()
2. scanf()
3. sprintf()
4. sscanf()

printf():

[printf\(\) function](#) is used in a C program to display any value like float, integer, character, string, etc on the console screen. It is a pre-defined function that is already declared in the stdio.h(header file).

Syntax 1:

To display any variable value.

printf("Format Specifier", var1, var2, ..., varn);

Example:

C

```
// C program to implement

// print f() function #include
<stdio.h>

// Driver code int
main()
{

    // Declaring an int type variable int a;

    // Assigning a value in a variable a = 20;

    // Print ing the value of a variable print f("%d", a);

    return 0;

}
```

Output

20

Syntax 2:

To display any string or a message

printf("Enter the text which you want to display");

Example:

C

```
// C program to implement

// printf() function #include
<stdio.h>

// Driver code int
main()
{

    // Displays the string written

    // inside the double quotes printf
    printf("This is a string"); return 0;
}
```

Output

This is a string

scanf():

scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like int, float, character, string, and many more. This function is declared in `stdio.h` (header file), that's why it is also a pre-defined function. In `scanf()` function we use `&` (address-of operator) which is used to store the variable value on the memory location of that variable.

Syntax:

scanf("Format Specifier", &var1, &var2, ..., &varn);

Example:

C

```
// C program to implement

// scanf() function
#include <stdio.h>


// Driver code int
main()
{

    int num1;


    // Print ing a message on
    // the output screen
    print f("Enter a int eger number: ");


    // Taking an int eger value
    // from keyboard
    scanf("%d", &num1);


    // Displaying the entered value print f("You
    have entered %d", num1);
```

```
    return 0;

}
```

Output

Enter a integer number: You have entered 0

Output:

Enter a integer number: 56 You have entered 56

sprintf():

[sprintf](#) stands for “**string print**”. This function is similar to printf() function but this function prints the string into a character array instead of printing it on the console screen.

Syntax:

sprintf(array_name, “format specifier”, variable_name);

Example:

C

// C program to implement

```
// the sprintf() function #include
<stdio.h>
```

```
// Driver code
int
main()
{
```

```
    char str[50];
```

```
    int a = 2, b = 8;
```

```
    // The string "2 and 8 are even number"
```



```
// is now stored into str

sprintf(str, "%d and %d are even number", a, b);

// Displays the string print
f("%s", str); return 0;
}
```

Output

2 and 8 are even number

sscanf():

[sscanf](#) stands for “**string scanf**”. This function is similar to scanf() function but this function reads data from the string or character array instead of the console screen.

Syntax:

sscanf(array_name, “format specifier”, &variable_name);

Example:

C

```
// C program to implement

// sscanf() function #include
<stdio.h>

// Driver code int
main()
{
```

```

charstr[50];

int a = 2, b = 8, c, d;


// The string "a = 2 and b = 8"
// is now stored into str
// character array
sprintf(str, "a = %d and b = %d", a, b);


// The value of a and b is now in
// c and d
sscanf(str, "a = %d and b = %d", &c,
        &d);


// Displays the value of c and d printf("c = %d
and d = %d", c, d); return 0;
}

```

Output

c = 2 and d = 8

Unformatted Input/Output functions

Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype. These functions are

used to read single input from the user at the console and it allows to display the value at the console.

Why they are called unformatted I/O?

These functions are called unformatted I/O functions because we cannot use format specifiers in these functions and hence, cannot format these functions according to our needs.

The following unformatted I/O functions will be discussed in this section-

1. `getch()`
2. `getche()`
3. `getchar()`
4. `putchar()`
5. `gets()`
6. `puts()`
7. `putch()`

`getch()`:

[`getch\(\)`](#) function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key. This function is declared in `conio.h`(header file). `getch()` is also used for hold the screen.

Syntax:

`getch(); or`

`variable-name = getch();`

Example:

C

```
// C program to implement
```

```
// getch() function
#include <conio.h>
#include <stdio.h>
```

```
// Driver code int
main()
{
```

```
print f("Enter any character: ");

// Reads a character but

// not displays
getch();

return 0;

}
```

Output:

Enter any character:

getche():

[getche\(\)](#) function reads a single character from the keyboard by the user and displays it on the console screen and immediately returns without pressing the enter key. This function is declared in conio.h(header file).

Syntax:

getche(); or

variable_name = getche();

Example:

C

```
// C program to implement

// the getche() function #include
<conio.h> #include <stdio.h>
```

```
// Driver code
int main()
{
    print f("Enter any character: ");

    // Reads a character and
    // displays immediately
    getch();
    return 0;
}
```

Output:

Enter any character: g

getchar():

The [getchar\(\)](#) function is used to read only a first single character from the keyboard whether multiple characters are typed by the user and this function reads one character at one time until and unless the enter key is pressed. This function is declared in `stdio.h` (header file)

Syntax:

Variable-name = getchar();

Example:

C

```
// C program to implement
// the getchar() function
#include <conio.h>
#include <stdio.h>
```

```
// Driver code int
main()
{

    // Declaring a char type variable charch;

    print f("Enter the character: ");

    // Taking a character from keyboard ch =
    getchar();

    // Displays the value of ch print
    f("%c", ch);
    return0;

}
```

Output:

Enter the character: a a

putchar():

The [putchar\(\)](#) function is used to display a single character at a time by passing that character directly to it or by passing a variable that has already stored a character. This function is declared in stdio.h(header file)

Syntax:

putchar(variable_name);

Example:**C**

```
// C program to implement

// the putchar() function
#include <conio.h> #include
<stdio.h>

// Driver code int
main()
{

    charch;

    print f("Enter any character: ");

    // Reads a character ch =
    getchar();

    // Displays that character
    putchar(ch);
    return0;

}
```

Output:*Enter any character: Z*

Z

gets():

gets() function reads a group of characters or strings from the keyboard by the user and these characters get stored in a character array. This function allows us to write space-separated texts or strings. This function is declared in stdio.h(header file).

Syntax:

char str[length of string in number]; //Declare a char type variable of any length

gets(str);

Example:

C

```
// C program to implement
```

```
// the gets() function #include
```

```
<conio.h> #include
```

```
<stdio.h>
```

```
// Driver code int
```

```
main()
```

```
{
```

```
    // Declaring a char type array
```

```
    // of length 50 characters char  
    name[50];
```

```
    print f("Please enter some texts: ");
```



```

// Reading a line of character or

// a string
gets(name);

// Displaying this line of character

// or a string

print f("You have entered: %s", name);
return0;

}

```

Output:

Please enter some texts: geeks for geeks You have entered: geeks for geeks

puts():

In C programming puts() function is used to display a group of characters or strings which is already stored in a character array. This function is declared in stdio.h(header file).

Syntax: *puts(identifier_name);*

Example:

```

// C program to implement

// the puts() function #include
<stdio.h>

// Driver code int
main()
{

    charname[50];

    print f("Enter your text: ");
    // Reads string from user gets(name);
    print f("Your text is: ");

    // Displays string
    puts(name);

    return0;

}

```

Output:

Enter your text: GeeksforGeeks Your text is: GeeksforGeeks

putch():

putch() function is used to display a single character which is given by the user and that character prints at the current cursor location. This function is declared in conio.h(header file)

Syntax:

putch(variable_name);

Example:

C

```
// C program to implement

// the putch() functions #include
<conio.h> #include <stdio.h>

// Driver code int
main()
{

    charch;

    printf("Enter any character:\n ");

    // Reads a character from the keyboard ch =
    getch();

    printf("\nEntered character is: ");

    // Displays that character on the console putch(ch);
    return 0;
```

```
}
```

Output:

*Enter any character:
Entered character is: d*

Formatted I/O vs Unformatted I/O

S No.	Formatted I/O functions	Unformatted I/O functions
1	These functions allow us to take input or display output in the user's desired format.	These functions do not allow to take input or display output in user desired format.
2	These functions support format specifiers.	These functions do not support format specifiers.
3	These are used for storing data more user friendly	These functions are not more user-friendly.
4	Here, we can use all data types.	Here, we can use only character and string data types.
5	printf(), scanf, sprintf() and sscanf() are examples of these functions.	getch(), getch(), gets() and puts(), are some examples of these functions.

String Manipulation Functions:

There are many important string functions defined in "string.h" library.

No.	Function	Description
------------	-----------------	--------------------

175

1)	<u>strlen(string_name)</u>	returns the length of string name.
2)	<u>strcpy(destination, source)</u>	copies the contents of source string to destination string.
3)	<u>strcat(first_string, second_string)</u>	concatenates or joins first string with second string. The result of string.
4)	<u>strcmp(first_string, second_string)</u>	compares the first string with second string. If both strings are same or not.
5)	<u>strrev(string)</u>	returns reverse string.

6)	<u>strlwr(string)</u>	returns string characters in lowercase.
7)	<u>strupr(string)</u>	returns string characters in uppercase.

User Defined Functions

Defination:

User-defined functions in C language are defined by the programmer to perform specific operations. They are also known as “***tailor-made functions***” which are built only to satisfy the given condition. We can classify functions into call by value or call by reference.

Advantages of using functions in the program are:

- One can avoid duplication of code in the programs by using functions. Code can be written more quickly and be more readable as a result.
- Code can be divided and conquered using functions. This process is known as Divide and Conquer. It is difficult to write large amounts of code within the main function, as well as testing and debugging. Our one task can be divided into several smaller sub-tasks by using functions, thus reducing the overall complexity.
- For example, when using pow, sqrt, etc. in C without knowing how it is implemented, one can hide implementation details with functions.
- With little to no modifications, functions developed in one program can be used in another, reducing the development time.

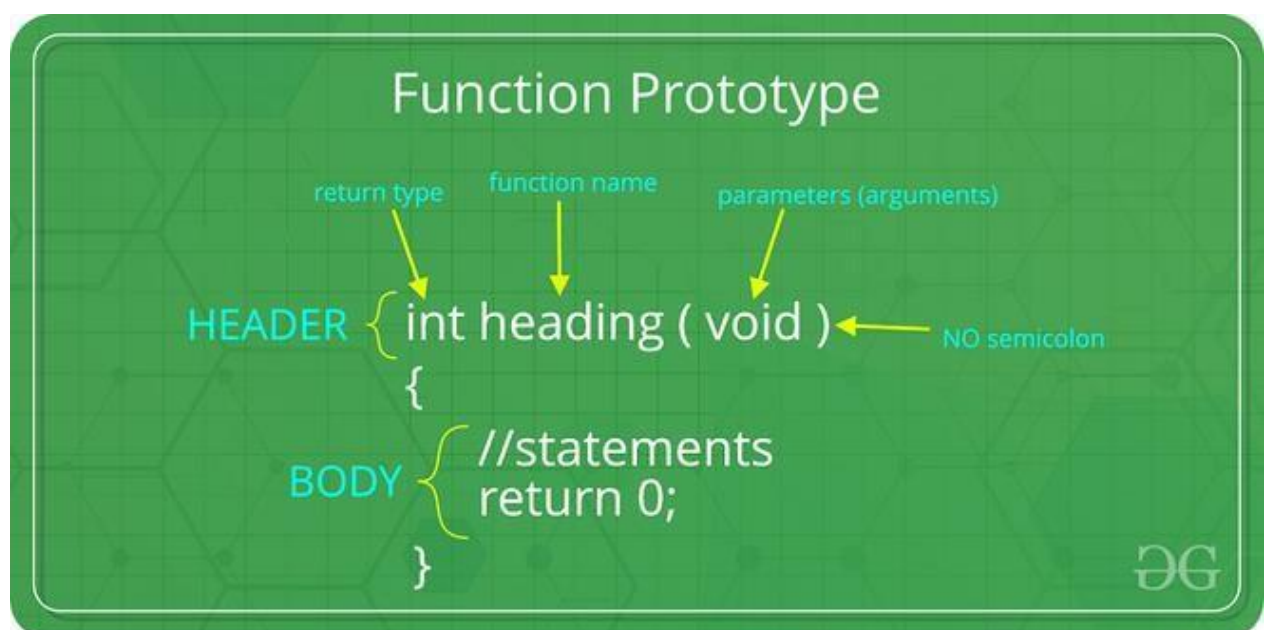
Elements of User-Defined Function in C

Function Prototype

A function prototype is also known as a function declaration which specifies the **function's name, function parameters, and return type**. The function prototype does not contain the body of the function. It is basically used to inform the compiler about the user-defined function which can be used in the later part of the program.

Syntax:

`return_type function_name(type1 arg1, type2 arg2, ... typeN argN);`



Function Declaration

Example:

```
int sum( int x, int y);
```

Here return_type of the function is an int eger and the name of the function is sum() which has two arguments of type int passed to the function.

Function Definition

Once the function has been called, the function definition contains the actual block of code that will be executed. The four components of a function definition are as follows:

- The return type of the function
- Function Name
- Function parameters
- Body of function

Function Calling

In order to transfer control to a user-defined function, we need to call it. Functions are called using their names followed by round brackets. Their arguments are passed inside the brackets. There are two types of function calls i.e, Call by value and Call by reference.

Syntax:

```
function_name(arg1, arg2, ... argN);
```

Example:

```
add(10,14);
```

A. Call by value

In call by value, a copy of the value is passed to the function and changes that are made to the function are not reflected back to the values. Actual and formal arguments are created in different memory locations.

Example:

C

```
// C program to show use of
```

```
// call by value #include
<stdio.h>
```

```
void swap(int a, int b)
```

```
{
```

```
    int temp = a;
```

```
    a = b;
```

```
    b = temp;
```

```
}
```

```
// Driver code int
```

```
main()
```

```
{
```

```
    int x = 10, y = 20;
```

```
    print f("Values of x and y before swap are: %d, %d\n", x, y); swap(x, y);
```

```
    print f("Values of x and y after swap are: %d, %d", x, y); return 0;
}
```

Output

Values of x and y before swap are: 10, 20 Values of x and y
after swap are: 10, 20

Note:

Values aren't changed in the call by value since they aren't passed by reference.

B. Call by Reference

In a call by Reference, the address of the argument is passed to the function, and changes that are made to the function are reflected back to the values.

Example:

```
// C program to implement
```

```
// Call by Reference #include
<stdio.h>
```

```
void swap(int *a, int *b)
```

```
{
```

```
    int temp = *a;
```

```
    *a = *b;
```

```
    *b = temp;
```

```
}
```

```
// Driver code int
```

```
main()
```

```
{
```

```
    int x = 10, y = 20;
```

```
    print f("Values of x and y before swap are: %d, %d\n", x, y); swap(&x, &y);
```

```
    print f("Values of x and y after swap are: %d, %d", x, y); return 0;
```

```
}
```

Output

Values of x and y before swap are: 10, 20 Values of x and y
after swap are: 20, 10

Return Statement

Return Statement is used to return value from the function. It basically terminates the execution of the function and the control of the program is returned to its caller. After the function call, the execution starts immediately. The return data type must match the defined data type in the function declaration and definition.

Syntax:

return (Expression)

Example:

return x+y;

Below is the program to demonstrate a user-defined function sum that returns the sum of arguments passed to it.

C

```
// C Program to show user-defined function
```

```
#include <stdio.h>
```

```
// Function declaration int
```

```
sum(int , int );
```

```
// Function definition int
```

```
sum(int x, int y)
```

```
{
```

```
    return x + y;
```

```
}
```

```
// Driver code int
```

```
main()
```

```
{
```

```
    int x = 10, y = 11;
```

```
    // Function call
```

```
    print f("Sum is %d ", sum(x, y));
```

```
    return 0;
```

```
}
```

Output

Sum is 21

Local and Global Variables:

What is a Global Variable?

- Global variables are those variables which are declared outside of all the functions or block and can be accessed globally in a program.
- It can be accessed by any function present in the program.
- Once we declare a global variable, its value can be varied as used with different functions.
- The lifetime of the global variable exists till the program executes. These variables are stored in fixed memory locations given by the compiler and do not automatically clean up.
- Global variables are mostly used in programming and useful for cases where all the functions need to access the same data.

Example:

```

1. #include<stdio.h>
2. int a=50, b=40;
3. void main()
4. {
5.     print f("a = %d and b=%d",a,b);
6. }

```

In the above example, a and b are the global variables.

Advantages of Global Variable

- Global variables can be accessed by all the functions present in the program.
- Only a single declaration is required.
- Very useful if all the functions are accessing the same data.

Disadvantages of Global Variable

- The value of a global variable can be changed accidentally as it can be used by any function in the program.
- If we use a large number of global variables, then there is a high chance of error generation in the program.

What is a Local Variable?

- **Variables that are declared within or inside a function block are known as Local variables.**
- These variables can only be accessed within the function in which they are declared.
- The lifetime of the local variable is within its function only, which means the variable exists till the function executes. Once function execution is completed, local variables are destroyed and no longer exist outside the function.
- The reason for the limited scope of local variables is that local variables are stored in the stack, which is dynamic in nature and automatically cleans up the data stored within it.
- But by making the variable static with "static" keyword, we can retain the value of local variable.

Example:

```

1. #include<stdio.h>
2. void main()
3. {
4.     int x=50, y=40;
5.     print f("x = %d and y=%d",x, y);
6. }

```

In the above example, we have declared x and y two variables inside the main function. Hence these are local variables.

Advantages of Local Variable

- The same name of a local variable can be used in different functions as it is only recognized by the function in which it is declared.
- Local variables use memory only for the limited time when the function is executed; after that same memory location can be reused.

Disadvantages of Local Variables

- The scope of the local variable is limited to its function only and cannot be used by other functions.
- Data sharing by the local variable is not allowed.

Comparison Chart Between Global Variable and Local Variable

Global Variable	Local Variable
Global variables are declared outside all the function blocks.	Local Variables are declared within a function
The scope remains throughout the program.	The scope is limited and remains within the are declared.
Any change in global variable affects the whole program, wherever it is being used.	Any change in the local variable does not a program.
A global variable exists in the program for the entire time the program is executed.	A local variable is created when the function execution is finished, the variable is destroyed
It can be accessed throughout the program by all the functions present in the program.	It can only be accessed by the function state and not by the other functions.
If the global variable is not initialized, it takes zero by default.	If the local variable is not initialized, it takes t
Global variables are stored in the data segment of memory.	Local variables are stored in a stack in memor
We cannot declare many variables with the same name.	We can declare various variables with the functions.

Examples to understand differences between Local and Global Variable
Now let's understand examples in different programming languages to better understand the difference between local and global variables.

Local vs Global in C

Example-1:

```
1. #include<stdio.h>
2.
3. // Global variables
4. int a;
5. int b;
6. int Add()
7. {
8.
9.     return a + b;
10. }
11.
12. int Mul()
13. {
14.
15.     int c=10; //Local Variable
16.     int d=20; ///Local Variable
17.     return c*d;
18. }
19. void main()
20. {
21.     int Ans1, Ans2, c=30; // Local variable
22.     a = 50;
23.     b = 70;
24.
25.     Ans1 = Add();
26.     Ans2= Mul();
27.
28.     print f("The addition result is: %d\n",Ans1);
29.     print f("The Multiplication result is: %d\n",Ans2);
30.     print f("%d\n", c);
31. }
```

Output:

```
The addition result is: 120
The Multiplication result is: 200
30
```

As we can see in the above program, we have taken **a** and **b** global variables that are being accessed in different functions such as Add() and main(). Whereas there are also local variables such as c, d, Ans1, and Ans2, which are being accessed by those functions only in which they are declared.

If we try to use c and d variables outside the Mul() function, they will be used as new variables. As we have shown by taking c in the main() function also, it is treated as a new variable.

Passing Parameters:

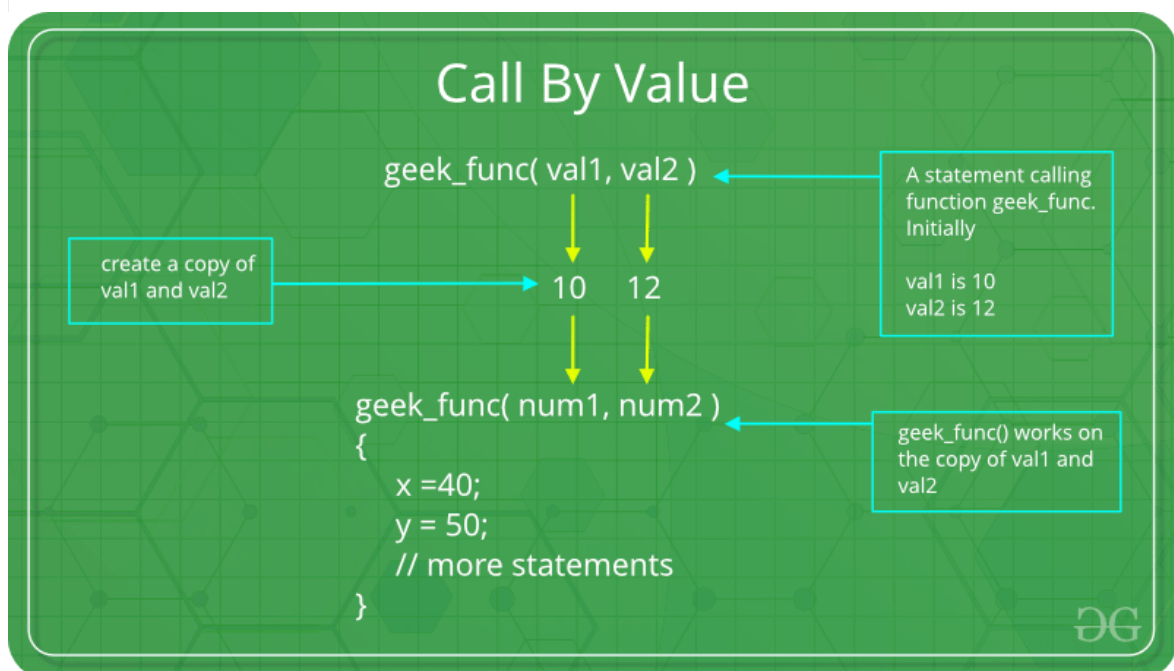
There are different ways in which parameter data can be passed into and out of methods and functions. Let us assume that a function B() is called from another function A(). In this case A is called the “**caller function**” and B is called the “**called function or callee function**”. Also, the arguments which A sends to B are called *actual arguments* and the parameters of B are called *formal arguments*.

Terminology

- **Formal Parameter :** A variable and its type as they appear in the prototype of the function or method.
- **Actual Parameter :** The variable or expression corresponding to a formal parameter that appears in the function or method call in the calling environment.
- **Modes:**
 - **IN:** Passes info from caller to callee.
 - **OUT:** Callee writes values in caller.
 - **IN/OUT:** Caller tells callee value of variable, which may be updated by callee.

Important methods of Parameter Passing

1. **Pass By Value:** This method uses *in-mode* semantics. Changes made to formal parameter do not get transmitted back to the caller. Any modifications to the formal parameter variable inside the called function or method affect only the separate storage location and will not be reflected in the actual parameter in the calling environment. This method is also called as **call by value**.



// C program to illustrate

```
// call by value #include
<stdio.h>
```

```
voidfunc(int a, int b)
```

```
{
    a += b;

    print f("In func, a = %d b = %d\n", a, b);
}

Int main(void)
{
    int x = 5, y = 7;

    // Passing parameters func(x, y);
    print f("In main, x = %d y = %d\n", x, y); return0;
}
```

CPP

```
// C++ program to illustrate
// call by value
#include <iostream>
namespacestd;
void func(int a, int b)
{
    a += b;
    cout <<"In func, a = "<< a << " b = "<< b << endl;
}

int main(void)
{
    int x = 5, y = 7;

    // Passing parameters func(x, y);
    cout << "In main, x = "<< x << " y = "<< y; return0;
}
```

Output:

In func, a = 12 b = 7 In main,
x = 5 y = 7

Languages like C, C++, Java support this type of parameter passing.

Shortcomings:

- Inefficiency in storage allocation
 - For objects and arrays, the copy semantics are costly
2. **Pass by reference(aliasing):** This technique uses *in/out-mode* semantics. Changes made to formal parameter do get transmitted back to the caller through parameter passing. Any changes to the formal parameter are reflected in the actual parameter in the calling environment as formal parameter receives a reference (or pointer) to the actual data. This method is also called as **call by reference**. This method is efficient in both time and space.

Call By Reference

```
void main()
```

```
{
```

```
  int n = 10;
```

```
  func(&n);
```

```
  printf( "%d", n );
```

```
}
```

```
void func ( int *add)
```

```
{
```

```
  *add = 20;
```

```
}
```

n

10 20
#2008

add

2008
#4016

Actual value of n
changes to 20



```
// C program to illustrate
// call by reference #include <stdio.h>
void swapnum(int * i, int * j)
{
    int temp = *i;
    *i = *j;

    *j = temp;
}

int main(void)
{
    int a = 10, b = 20;

    // passing
    parameter
```

```
s
swapnum
(&a, &b);

print f("a is %d and b is
%d\n", a, b);
return 0;
}
```

1. C and C++ both support call by value as well as call by reference whereas Java doesn't support call by reference.

Shortcomings:

- Many potential scenarios can occur
- Programs are difficult to understand sometimes

Other methods of Parameter Passing

These techniques are older and were used in earlier programming languages like Pascal, Algol and Fortran. These techniques are not applicable in high level languages.

1. **Pass by Result:** This method uses *out-mode* semantics. Just before control is transferred back to the caller, the value of the formal parameter is transmitted back to the actual parameter. This method is sometimes called *call by result*. In general, pass by result technique is implemented by copy.
2. **Pass by Value-Result:** This method uses *in/out-mode* semantics. It is a combination of Pass-by-Value and Pass-by-result. Just before the control

is transferred back to the caller, the value of the formal parameter is transmitted back to the actual parameter. This method is sometimes called as *call by value-result*

3. **Pass by name :** This technique is used in programming language such as *Algol*. In this technique, symbolic "*name*" of a variable is passed, which allows it both to be accessed and update.

Example:

To double the value of $C[j]$, you can pass its name (not its value) into the following procedure.

4. procedure double(x);
5. real x;
6. begin
7. $x := x * 2$
8. end;

In general, the effect of pass-by-name is to textually substitute the argument in a procedure call for the corresponding parameter in the body of the procedure.

Implications of Pass-by-Name mechanism:

- The argument expression is re-evaluated each time the formal parameter is passed.
- The procedure can change the values of variables used in the argument expression and hence change the expression's value.

Recursion:

Recursion is the process of repeating items in a self-similar way. In programming languages, if a program allows you to call a function inside the same function, then it is called a recursive call of the function.

```
void recursion() {
    recursion(); /* function calls itself */
}

int main() {
    recursion();
}
```

The C programming language supports recursion, i.e., a function to call itself. But while using recursion, programmers need to be careful to define an exit condition from the function, otherwise it will go into an infinite loop.

Recursive functions are very useful to solve many mathematical problems, such as calculating the factorial of a number, generating Fibonacci series, etc.

Number Factorial

The following example calculates the factorial of a given number using a recursive function —

```
#include <stdio.h>

unsigned long long int factorial(unsigned int i) {

    if(i <= 1) {
        return 1;
    }
    return i * factorial(i - 1);
}

int main() {
    int i = 12;
    printf("Factorial of %d is %d\n", i, factorial(i));
    return 0;
}
```

When the above code is compiled and executed, it produces the following result —

Factorial of 12 is 479001600

Fibonacci Series

The following example generates the Fibonacci series for a given number using a recursive function —

```
#include <stdio.h>

int fibonacci(int i) {

    if(i == 0) {
        return 0;
    }

    if(i == 1) {
        return 1;
    }
    return fibonacci(i-1) + fibonacci(i-2);
}

int main() {

    int i;

    for (i = 0; i < 10; i++) {
        printf("%d\t\n", fibonacci(i));
    }

    return 0;
}
```

When the above code is compiled and executed, it produces the following result –

```
0
1
1
2
3
5
8
13
21
34
```